

Dokumentation der Feldlerchen-Singflug-Generierung

Dieses Kapitel beschreibt die Funktion `generate_skylarks` aus dem Modul `skylark_generation.py`. Die Funktion erzeugt simulierte ENU-Positionsdaten für Feldlerchen (`lark_XXXX`), die typische Singflüge um ein Revier- bzw. Nestzentrum ausführen.

Ein Flug besteht aus drei Phasen:

- Steigflug mit zunehmend größer werdenden Kreisen
- Kreisförmiger Singflug in größerer Höhe
- Spiralförmiger Sinkflug zurück in Richtung Nestzentrum

Kernfunktion: Feldlerchen generieren (`generate_skylarks`)

Die Funktion nimmt räumliche Grenzen, Simulationsparameter und einen optionalen Zufalls-Seed entgegen. Sie gibt eine Liste von Positionsdatensätzen zurück. Jeder Datensatz enthält eine Vogel-ID, einen Zeitstempel sowie Ost-, Nord- und Höhenkoordinaten im ENU-System.

1. Eingabeprüfung und Zufallsinitialisierung

Zu Beginn prüft die Funktion die übergebenen Parameter. Bei ungültigen Werten wird jeweils ein `ValueError` ausgelöst.

Folgende Bedingungen müssen erfüllt sein:

- `number_of_larks` darf nicht negativ sein.
- `time_interval` muss größer als 0 sein.
- `x_min` darf nicht größer als `x_max` sein.
- `y_min` darf nicht größer als `y_max` sein.
- `z_max` muss größer als `z_min` sein.
- `position_noise` darf nicht negativ sein.
- Die minimale Flugdauer muss größer als 0 und kleiner oder gleich der maximalen Flugdauer sein.
- Die minimale Singflughöhe muss größer als 0 und kleiner oder gleich der maximalen Singflughöhe sein.

Wird `number_of_larks` mit 0 übergeben, gibt die Funktion direkt eine leere Liste zurück. In diesem Fall werden die übrigen Parameter nicht weiter geprüft.

Für die Zufallswerte wird ein separates Zufallsobjekt verwendet:

- Bei `seed_value = None` wird ein nicht deterministischer Zufallszahlengenerator erzeugt.
- Bei einem gesetzten Seed wird aus dem Wert `"{seed_value}:skylarks"` ein SHA-256-Hash berechnet.
- Die ersten acht Bytes dieses Hashes werden als Seed verwendet.

Dadurch erhalten Feldlerchen einen eigenen, reproduzierbaren Zufallsstrom. Änderungen an der Feldlerchen-Generierung beeinflussen somit nicht die Zufallswerte anderer simulierten Vogeltypen.

2. Bestimmung von Nestzentrum und Flugprofil

Für jede Feldlerche wird zunächst ein individuelles Nest- bzw. Revierzentrum bestimmt.

A. Horizontale Flugausdehnung:

Die maximal mögliche Kreisflugradius wird anhand der verfügbaren X- und Y-Grenzen berechnet:

```
horizontal_radius_limit = max(0, min(70.0, (x_max - x_min) / 2 - 3 *
position_noise, (y_max - y_min) / 2 - 3 * position_noise))
```

Der Radius ist dabei auf maximal 70 Einheiten begrenzt. Zusätzlich werden drei Standardabweichungen des Positionsrauschens als Sicherheitsabstand zu den horizontalen Grenzen berücksichtigt.

Das Nestzentrum wird möglichst so gewählt, dass Kreisflug und Positionsrauschen innerhalb des Simulationsbereichs liegen. Ist der verfügbare Bereich zu klein für den erforderlichen Randabstand, wird das Nestzentrum auf die Mitte des jeweiligen Bereichs gesetzt.

Der tatsächliche Kreisflugradius wird zufällig zwischen 55 % und 85 % des berechneten Maximalradius gewählt.

B. Start- und Singflughöhe:

Die Startposition liegt knapp über der unteren Höhenbegrenzung:

- Die Startflughöhe liegt maximal 5 Einheiten oder 10 % des vertikalen Bereichs über `z_min`.
- Die maximale Flughöhe wird ausgehend von dieser Startflughöhe bestimmt.

Wenn ausreichend vertikaler Raum vorhanden ist, wird der Höhengewinn zufällig zwischen `min_song_height` und `max_song_height` gewählt. Dabei bleibt die maximale Höhe bei höchstens 95 % des verfügbaren Höhenraums.

Ist der Simulationsraum kleiner als die gewünschte Mindest-Singflughöhe, wird stattdessen ein Höhengewinn zwischen 60 % und 90 % des verfügbaren Höhenraums verwendet.

Zusätzlich wird eine leichte vertikale Schwankung berechnet. Diese beträgt maximal 4 Einheiten und simuliert natürliche Höhenänderungen während des Singflugs.

C. Flugdauer und Zeitstempel:

Die Flugdauer jeder Feldlerche wird zufällig zwischen `min_flight_duration` und `max_flight_duration` gewählt.

Die Anzahl der erzeugten Positionsmessungen wird wie folgt bestimmt:

```
number_of_samples = max(3, round(duration / time_interval) + 1)
```

Jeder Flug enthält somit mindestens drei Messpunkte. Die tatsächlich abgebildete Dauer ergibt sich aus:

```
(number_of_samples - 1) * time_interval
```

Sie kann durch die Rundung leicht von der ursprünglich gezogenen Flugdauer abweichen.

Der Startzeitpunkt wird aus dem Datum von `sim_date` gebildet. Die Uhrzeit wird zufällig zwischen 05:00 Uhr und 19:59 Uhr gewählt. Die einzelnen Messungen folgen anschließend im Abstand von `time_interval` Sekunden.

3. Flugphasen und Positionsrechnung

Jede Feldlerche durchläuft drei Flugphasen. Der Fortschritt eines Messpunkts innerhalb des Flugs wird als Wert zwischen 0 und 1 berechnet:

```
phase = sample_index / (number_of_samples - 1)
```

Die Übergänge zwischen den Phasen werden mit einer Cosinus-Glättung erzeugt:

```
ease(progress) = 0.5 - 0.5 * cos(pi * progress)
```

Diese Glättung sorgt dafür, dass Aufstieg und Abstieg nicht abrupt beginnen oder enden.

A. Steigflug:

Der Steigflug endet zufällig zwischen 26 % und 36 % der gesamten Flugdauer.

Während dieser Phase:

1. steigt die Feldlerche von der Startflughöhe zur maximalen Flughöhe auf,
2. wächst der Abstand zum Nestzentrum gleichmäßig und geglättet an,
3. entsteht durch die zunehmende Kreisbewegung eine aufsteigende Spiralförmigkeit.

B. Kreis- und Singflug:

Der Singflug endet zufällig zwischen 62 % und 74 % der gesamten Flugdauer.

Während dieser Phase:

1. bleibt der horizontale Radius konstant,
2. kreist die Feldlerche um das Nestzentrum,
3. bleibt die Flughöhe überwiegend nahe der maximalen Höhe,
4. erzeugen sinusförmige Schwankungen leichte natürliche Höhenbewegungen.

C. Sinkflug:

Nach dem Singflug beginnt der spiralförmige Abstieg.

Während dieser Phase:

1. nimmt die Flughöhe geglättet wieder bis zur Startflughöhe ab,
2. verkleinert sich der Kreisradius bis auf 0,
3. nähert sich die Feldlerche wieder dem Nestzentrum,
4. wird weiterhin eine reduzierte vertikale Schwankung simuliert.

Die Drehrichtung wird pro Feldlerche zufällig gewählt. Der Flug kann somit im oder gegen den Uhrzeigersinn verlaufen. Die Anzahl der vollständigen Kreisbewegungen liegt zufällig zwischen 2,5 und 5,0 Umdrehungen.

4. Positionsrauschen und Begrenzung

Für jeden Messpunkt werden die theoretischen X-, Y- und Z-Koordinaten berechnet.

Anschließend wird Positionsrauschen hinzugefügt:

- Für `enu_e` und `enu_n` wird gaußförmiges Rauschen mit der Standardabweichung `position_noise` verwendet.
- Für `enu_u` wird unabhängig davon ein geringes gaußförmiges Rauschen mit einer Standardabweichung von `0.03` verwendet.

Nach dem Hinzufügen des Rauschens werden alle Koordinaten auf die vorgegebenen Grenzen begrenzt:

- `enu_e` auf den Bereich von `x_min` bis `x_max`
- `enu_n` auf den Bereich von `y_min` bis `y_max`
- `enu_u` auf den Bereich von `z_min` bis `z_max`

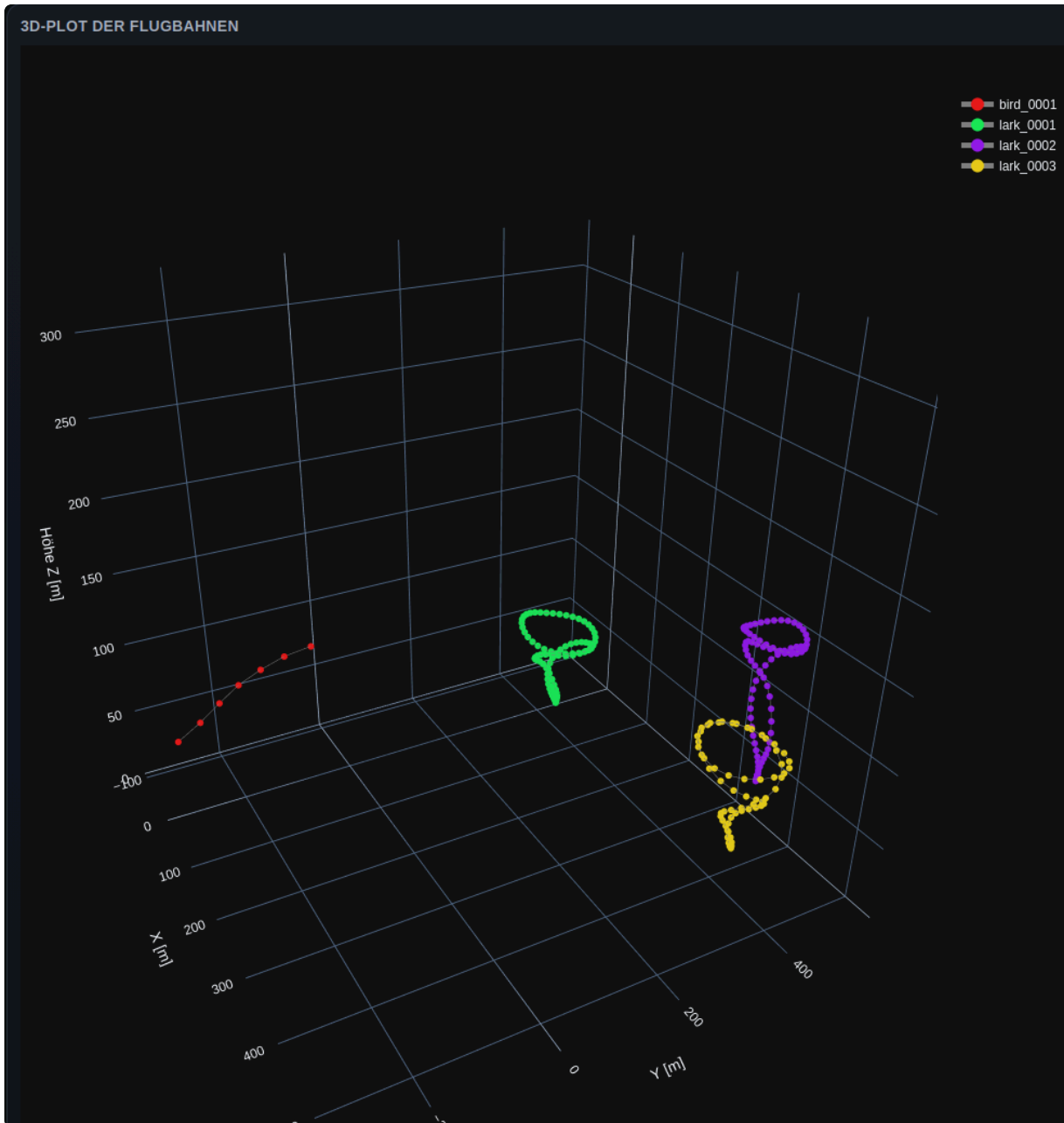
Die Koordinaten werden anschließend auf zwei Nachkommastellen gerundet.

5. Rückgabe der Datensätze

Für jede Feldlerche wird eine eindeutige ID im Format `lark_XXXX` erzeugt, beispielsweise `lark_0001`.

Die Funktion gibt alle erzeugten Messpunkte als Liste von Dictionaries zurück. Die Datensätze sind nach Feldlerche und innerhalb einer Feldlerche nach Messzeitpunkt angeordnet. Die gesamte Rückgabeliste ist nicht zwingend global nach Zeitstempel sortiert.

Werden die einzelnen Messpunkte geplottet, kann man eindeutig den Singflug der Feldlerche erkennen:



Hilfsfunktionen

`clamp`

Begrenzt einen Wert auf einen definierten unteren und oberen Grenzwert.

Ein Wert unterhalb der unteren Grenze wird auf die untere Grenze gesetzt. Ein Wert oberhalb der oberen Grenze wird auf die obere Grenze gesetzt.

ease

Berechnet einen geglätteten Fortschrittswert zwischen 0 und 1 mittels einer Cosinusfunktion.

Die Funktion wird für sanfte Übergänge beim Aufstieg und beim Abstieg verwendet.

random_center

Erzeugt eine zufällige Mittelpunktcoordinate innerhalb eines Bereichs unter Berücksichtigung eines Sicherheitsabstands.

Ist der verfügbare Bereich kleiner als der doppelte Sicherheitsabstand, wird stattdessen die Mitte des Bereichs zurückgegeben.

Parameter und Rückgabewert (generate_skylarks)

Parameter	Standardwert	Beschreibung
number_of_larks	–	Anzahl der zu generierenden Feldlerchen. Muss größer oder gleich 0 sein.
time_interval	–	Zeitlicher Abstand zwischen zwei Positionsmessungen in Sekunden. Muss größer als 0 sein.
sim_date	–	datetime -Objekt, dessen Datum für die Simulation verwendet wird.
x_min	–	Untere Grenze der Ostkoordinate (enu_e).
x_max	–	Obere Grenze der Ostkoordinate (enu_e).
y_min	–	Untere Grenze der Nordkoordinate (enu_n).
y_max	–	Obere Grenze der Nordkoordinate (enu_n).
z_min	–	Untere Grenze der Höhe (enu_u).
z_max	–	Obere Grenze der Höhe (enu_u). Muss größer als z_min sein.
position_noise	–	Standardabweichung des horizontalen Positionsrauschens für enu_e und enu_n .
seed_value	–	Optionaler Seed für reproduzierbare Zufallswerte. Bei None wird ein zufälliger Seed verwendet.
min_flight_duration	180	Minimale Flugdauer einer Feldlerche in Sekunden.

Parameter	Standardwert	Beschreibung
max_flight_duration	480	Maximale Flugdauer einer Feldlerche in Sekunden.
min_song_height	50.0	Minimaler Höhengewinn über der Startflughöhe, sofern der Luftraum ausreichend groß ist.
max_song_height	120.0	Maximaler Höhengewinn über der Startflughöhe, sofern der Luftraum ausreichend groß ist.

Struktur eines erzeugten Datensatzes

Feld	Beschreibung
bird_id	Eindeutige Feldlerchen-ID im Format lark_XXXX .
determined_bird_id	Platzhalterwert bird_0000 .
timestamp	Zeitstempel der Messung im ISO-8601-Format mit angehängtem Z .
enu_e	Ostkoordinate der simulierten Position.
enu_n	Nordkoordinate der simulierten Position.
enu_u	Höhenkoordinate der simulierten Position.